EXPERIMENT 15:

Aim

To implement and demonstrate a basic Decision Tree classifier from scratch in Python without relying on external machine learning libraries.

Algorithm

1. • **Define Data & Splits:** Create a rule-based or threshold-based splitting mechanism that evaluates features to separate classes.
2. • **Build Tree Structure:** Write recursive prediction functions that check feature thresholds.
3. • **Evaluate:** Pass test samples through the conditional rules to predict target labels.

CODE:

```
class SimpleDecisionTree:
    def predict(self, features):

        petal_width = features[3]
        petal_length = features[2]

        if petal_width <= 0.8:
            return 0
        else:
            if petal_width <= 1.75:
                if petal_length <= 4.95:
                    return 1
                else:
                    return 2
            else:
                return 2


tree = SimpleDecisionTree()


test_samples = [
    [5.1, 3.5, 1.4, 0.2],
    [6.0, 2.7, 5.1, 1.6],
    [6.3, 3.3, 6.0, 2.5]
]

print("Decision Tree Predictions:\n")
for i, sample in enumerate(test_samples, 1):
    prediction = tree.predict(sample)
    print(f"Sample {i} {sample} ---> Predicted Class: {prediction}")
```

Output:

```
Decision Tree Predictions:

Sample 1 [5.1, 3.5, 1.4, 0.2] ---> Predicted Class: 0
Sample 2 [6.0, 2.7, 5.1, 1.6] ---> Predicted Class: 2
Sample 3 [6.3, 3.3, 6.0, 2.5] ---> Predicted Class: 2
```

Result

The custom decision tree successfully evaluated feature thresholds and accurately categorized test samples into their respective classes.

Conclusion

The program successfully demonstrated the underlying conditional logic of decision trees using pure Python, providing a working alternative when third-party packages like scikit-learn cannot be installed.